IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Build Any Robust Multidisciplinary Product with AI Agents

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—This paper presents Fiducial, an open-source platform for building products that combine software and hardware with AI coding agents. Every fact about a product is declared once in typed text files, every artifact that uses it – code, translations, circuit board, case – is derived from it by one command, and checks on every commit refuse anything that no longer matches its declaration. In a test of 20 realistic mistakes in a web product and a USB-C sensor stick, the first version caught 15. Each miss was turned into a general rule, and the current version catches all 20.**

***Keywords*—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration**

I. Introduction

While building Outreach [1], an off-grid messaging system made of radios, gateways, a companion app and printed cases, the author met the same problem more than once. The firmware and the app disagreed on the layout of a message, and a printed case did not fit the board it was made for. Neither part was wrong on its own. The facts they shared had been copied by hand between different tools, one copy changed, and the mistake showed up only when the parts were put together.

AI coding agents [2], [3] now write much of this work, but they make the problem faster rather than smaller: an agent that knows one codebase does not know what changed in the others. Every project also has its own layout and habits, so each new product starts from scratch.

Fiducial [4] gives every product the same structure, built on one rule: each fact is declared once, everything that uses it is derived from it, and every change is checked (Fig. 1). People and agents edit the declarations, never the outputs. The idea builds on model-driven engineering [5] and on build systems that track their inputs by hash [6], applied to every discipline of one product at once, in plain files that agents read and write well.

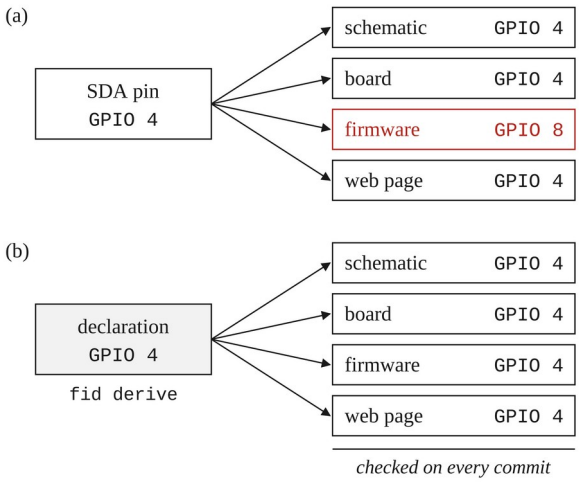


Fig. 1. One fact in four places: (a) copied by hand, one copy changes and nothing notices; (b) declared once, every copy is derived and checked.

II. System Overview and Implementation

Fiducial is a command-line tool, fid, written in Rust. fid new creates a product repository with a fixed layout, and fid add enables capabilities – packaged pipelines for web applications, translations, legal pages, design systems, databases, firmware, hardware and more. Because every product shares one structure, a new one starts from working, checked parts. Fig. 2 shows how a change moves through it.

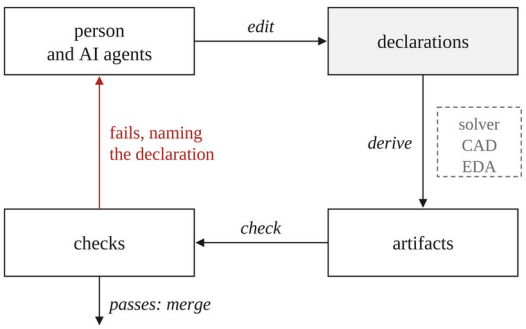


Fig. 2. A change in Fiducial: declarations are edited, derivation produces the artifacts with help from established engines, and the checks either accept the change or name the declaration at fault.

A. *Declarations*

A declaration is a typed text file in version control: the strings of each language, the brand and pages, the colour palette, the messages a device sends, and the parts of a physical product with what each pin connects to. Text files were chosen over a modelling tool because coding agents read and write them well, and every change goes through a normal code review. A declaration holds facts only; nothing in it says where a part sits on the board or how a page is laid out.

B. *Derivation*

fid derive runs every pipeline. Each one reads the declarations it needs and writes its artifacts, such as a typed translation module, a sitemap, a firmware pin map, TypeScript types or a KiCad board [7]. The hash of every input and output is stored in a lock file. Derivation is deterministic: the same inputs always give the same bytes, so a changed hash always means a real change.

C. *Engines for Hard Problems*

Some derivations are real engineering problems. The first version placed board parts with a hand-written search, and each product added special rules until they began to interact. Now the platform only writes the problem down and hands it to an established engine: the CP-SAT solver [8] places the parts, build123d on the OpenCascade kernel [9] builds the case, and KiCad routes and checks the board. Every placement rule is one of eight generic kinds, so a new requirement is a new declaration, not new code.

D. *Checks*

Three kinds of check run on every commit. fid derive --check fails when a declaration changed without its outputs being re-derived, or when an output was edited by hand. Derivation refuses declarations that break a rule: a missing translation, too little colour contrast, parts over the cost limit, a board that does not fit its case. Compilers and type checkers read the derived files, so code that still uses a renamed fact stops compiling. Each failure names the file and the declaration at fault, which is the signal an agent needs to correct itself.

E. *Working with AI Agents*

Every product carries an AGENTS.md file [10] and one instruction file per capability, so any agent starts from the same context. Agents are told to fix a problem where the fact is declared, never in a generated file. When a mistake gets past the checks, it is not patched for that one product; it becomes a general rule for all of them.

F. *Worked Example: Sensor Stick*

The repository includes a public example: a USB-C stick that reports temperature and humidity to a web page (Fig. 3). It has a printed case, a two-layer board with 23 parts, Rust firmware and a web page. The person writes the product declaration, an outline drawing, the firmware and the page. Fiducial derives the board, the case, the manufacturing files, and the code both sides use to read the sensor's messages.

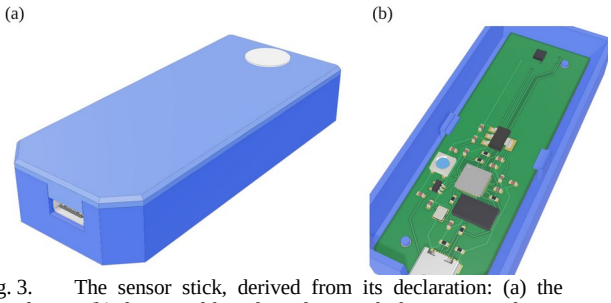


Fig. 3. The sensor stick, derived from its declaration: (a) the closed case; (b) the routed board inside it, with the sensor under its vent.

III. Results

Fiducial was built in six weeks by one person working with AI agents and used on two products, the sensor stick and a private one. The sensor stick's board passes KiCad's design-rule check with no errors. Moving its sensor to another pin is one changed line: fid derive then rewrites 8 generated files in 37 s, including the board, the firmware's pin map and the page's types, and neither the firmware nor the page needs an edit.

To test whether the checks catch the mistakes people actually make, 20 realistic mistakes were injected one at a time, in the manner of mutation testing [11]: ten in a fresh web product in English and Serbian, and ten in the sensor stick's hardware, firmware and interface. They cover eight kinds of drift (Table I). After each one the checks ran as in continuous integration, and the product was reset. Three correct changes and two runs with no change confirmed that the checks do not fail without a reason. The test is a script in the repository.

TABLE I. Injected Mistakes, by Kind of Drift

Kind of drift	**Example**	**0.9.4**	**Now**
Stale output	Domain edited, not re-derived	1/3	3/3
Generated file edited	Pin map edited by hand	5/5	5/5
Missing declaration	String in English only	3/3	3/3
Rule broken	Contrast below 4.5 : 1	3/3	3/3
Declarations conflict	Case too narrow for board	2/2	2/2
Code out of date	Net renamed, old name used	1/1	1/1
Declaration bypassed	Pin typed as a number	0/2	2/2
Not translated	Serbian left in English	0/1	1/1
Total		15/20	20/20

On release 0.9.4 the checks caught 15 of the 20. Each miss was a fact the platform did not yet own: two declarations edited without re-deriving went unnoticed, because only the hardware pipeline tracked its inputs; firmware could name a pin or a sensor address directly; and a Serbian string left in English passed every structural check. Each miss became a general rule. On release 0.9.11, all 20 are caught – 10 during derivation, 9 by --check and 1 by the compiler – and every correct change passes. Fig. 4 shows two of the messages.

(a)

```
$ fid derive
fid-i18n: the catalogs are not complete.
messages/sr.json is missing 1 key(s):
action.delete
```

(b)

```
$ fid derive
firmware types what the board already declares (a pin,
an address, a command or a wait):
firmware/rp2040/src/main.rs:84: takes PIN_4 directly —
take it through `board::<net>!(p)` …
```

Fig. 4. Two failures as the person or agent sees them: (a) a string added in English only; (b) firmware naming a pin directly.

The mistakes were chosen by the author of the platform, only two products were built, and neither has been manufactured yet. The test measures the checks, not whether agents make fewer mistakes with Fiducial than without it.

IV. Conclusion

Better models will move the line between what a person checks and what is left to agents, but a probabilistic system cannot guarantee its own output. Fiducial keeps that output bounded: each fact is declared once, everything else is derived, and every change is checked. With it, one person working with AI agents kept a product's code, board and case in agreement, and mistakes were caught early by messages that name their cause. Future work will manufacture and test the sensor stick and compare agents working with and without the platform on the same tasks. Fiducial and the example are open source [4].

V. References

[1] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[2] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[3] OpenAI Codex. Available at: https://openai.com/codex/

[4] A. Zdravković, Fiducial, release 0.9.11, 2026. Available at: https://github.com/AleksaZCodes/fiducial

[5] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006.

[6] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[7] KiCad EDA. Available at: https://www.kicad.org/

[8] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[9] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[10] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[11] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978.